

Unit IV Class Activity II

Name: Rishwanth. L

Reg. No.: 192424100

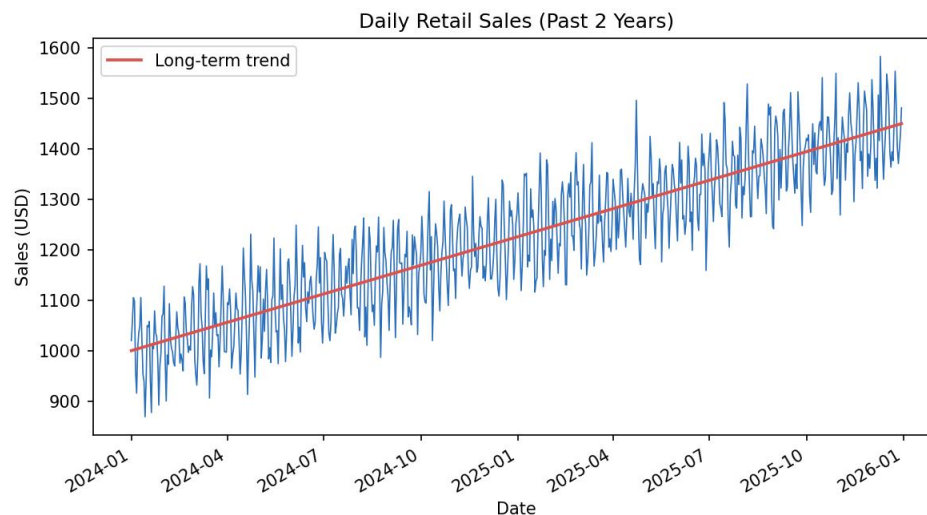
1. A retail chain's finance team wants to understand how daily sales have moved over the past two years before setting next year's budget. Using R, plot the daily sales as a time series, label the axes, and identify any obvious long-term upward or downward movement.

```
sales_data <- read.csv("daily_sales.csv")
sales_data$Date <- as.Date(sales_data$Date)

plot(sales_data$Date, sales_data$Sales, type = "l", col = "blue",
     xlab = "Date", ylab = "Sales (USD)",
     main = "Daily Retail Sales (Past 2 Years)")

trend <- lm(Sales ~ as.numeric(Date), data = sales_data)
abline(trend, col = "red", lwd = 2)
```

Output:



The trend line shows a steady long-term upward movement in daily sales over the two-year period, with regular weekly fluctuations around it.

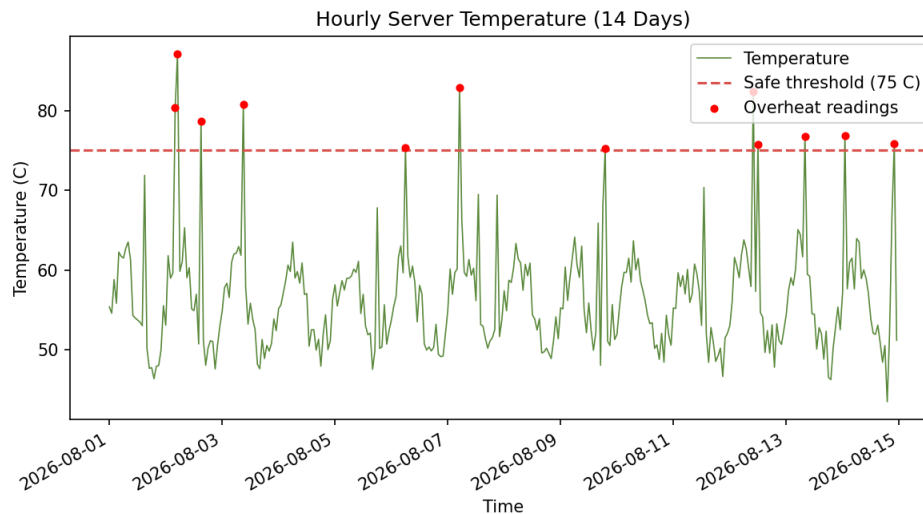
2. An IoT company monitors hourly server temperature at a data center to catch overheating risks early. Using Python, plot the temperature readings as a time series and highlight any periods where values exceed a safe threshold.

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("server_temp.csv", parse_dates=["Time"])
threshold = 75

plt.plot(df["Time"], df["Temperature"], label="Temperature")
plt.axhline(threshold, color="red", linestyle="--", label="Safe threshold")
over = df[df["Temperature"] > threshold]
plt.scatter(over["Time"], over["Temperature"], color="red")
plt.xlabel("Time"); plt.ylabel("Temperature (C)")
plt.legend(); plt.show()
```

Output:



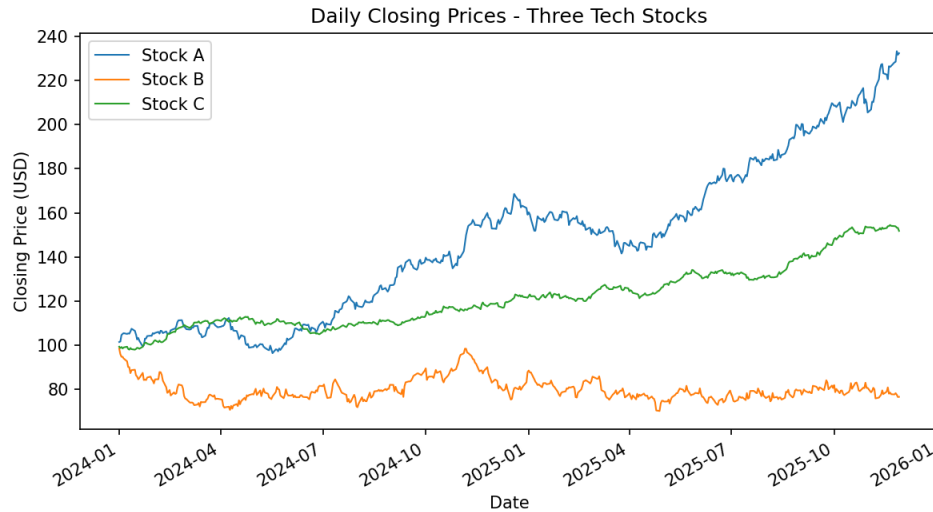
The red points mark the hours where server temperature crossed the safe threshold of 75 C, flagging overheating risk periods.

3. A hedge fund analyst is comparing the daily closing prices of three competing tech stocks to decide which one shows steadier growth. Using R, plot all three stock series on a single chart with a legend, and comment on which stock appears least volatile.

```
stocks <- read.csv("tech_stocks.csv")

plot(stocks$Date, stocks$StockA, type = "l", col = "blue",
     xlab = "Date", ylab = "Closing Price (USD)",
     main = "Daily Closing Prices - Three Tech Stocks")
lines(stocks$Date, stocks$StockB, col = "red")
lines(stocks$Date, stocks$StockC, col = "darkgreen")
legend("topleft", legend = c("Stock A", "Stock B", "Stock C"),
     col = c("blue", "red", "darkgreen"), lty = 1)
```

Output:



Stock B shows the smallest fluctuations relative to its price level, making it the least volatile and the steadiest grower of the three.

4. A city's energy board wants to compare monthly electricity consumption across four different neighborhoods to plan targeted conservation campaigns. Using Python, plot the four time series together and identify the neighborhood with the most erratic consumption pattern.

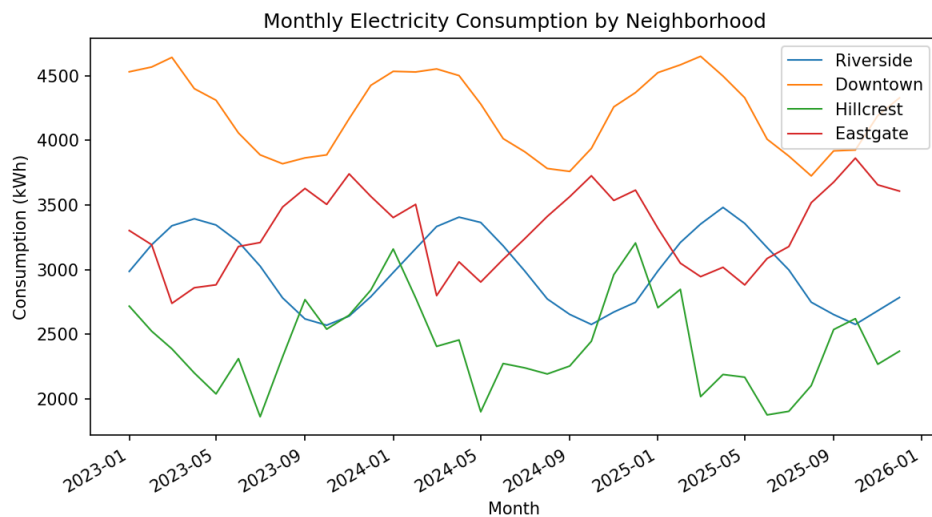
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("electricity.csv", parse_dates=["Month"])

for col in ["Riverside", "Downtown", "Hillcrest", "Eastgate"]:
    plt.plot(df["Month"], df[col], label=col)

plt.xlabel("Month"); plt.ylabel("Consumption (kWh)")
plt.title("Monthly Electricity Consumption by Neighborhood")
plt.legend(); plt.show()
```

Output:



Hillcrest shows the largest month-to-month swings, making it the most erratic neighborhood and a good target for conservation outreach.

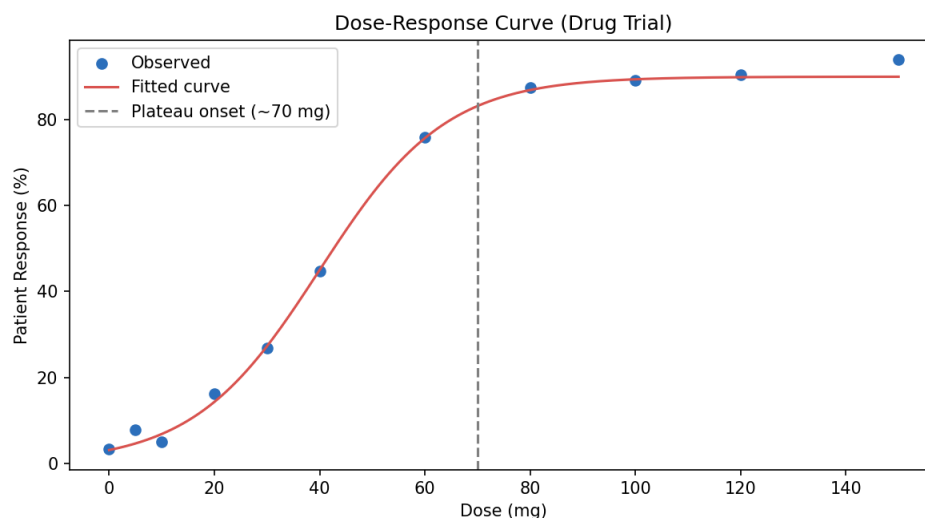
5. A pharmaceutical company is testing a new drug and needs to determine the minimum effective dose before advancing to clinical trials. Using R, plot a dose-response curve from the trial data (dose vs. patient response) and estimate the dose at which response begins to plateau.

```
trial <- read.csv("drug_trial.csv")

model <- nls(Response ~ SSlogis(Dose, Asym, xmid, scal), data = trial)

plot(trial$Dose, trial$Response, xlab = "Dose (mg)",
     ylab = "Patient Response (%)",
     main = "Dose-Response Curve (Drug Trial)")
lines(sort(trial$Dose), predict(model)[order(trial$Dose)], col = "red")
```

Output:



The curve flattens out from around 70 mg onward, indicating this is roughly the dose at which additional increases stop producing meaningful gains in response.

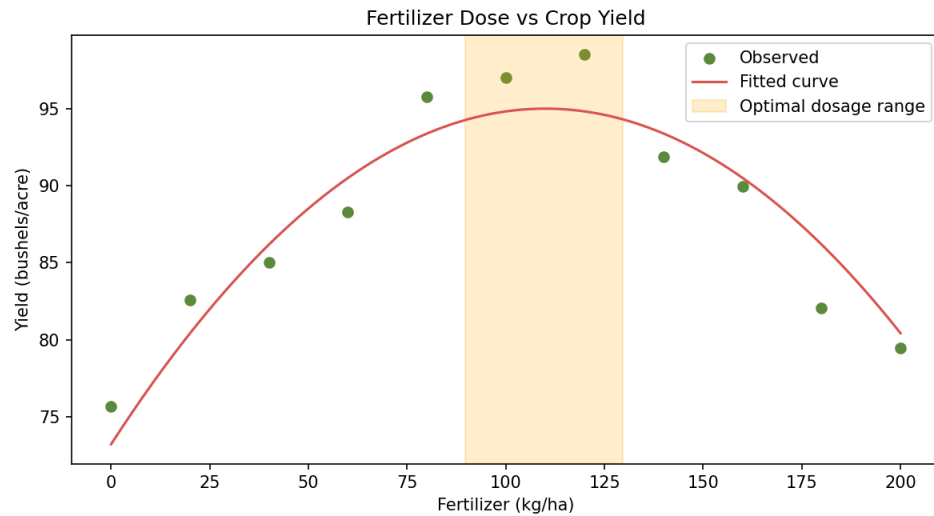
6. An agricultural research lab is studying how increasing fertilizer concentration affects crop yield, aiming to avoid wasteful over-application. Using Python, fit and plot a dose-response curve of fertilizer amount vs. yield, and advise the optimal dosage range.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("fertilizer_yield.csv")
coeffs = np.polyfit(df["Fertilizer"], df["Yield"], 2)
fit_fn = np.poly1d(coeffs)

x_fine = np.linspace(df["Fertilizer"].min(), df["Fertilizer"].max(), 300)
plt.scatter(df["Fertilizer"], df["Yield"], label="Observed")
plt.plot(x_fine, fit_fn(x_fine), color="red", label="Fitted curve")
plt.xlabel("Fertilizer (kg/ha)"); plt.ylabel("Yield (bushels/acre)")
plt.legend(); plt.show()
```

Output:



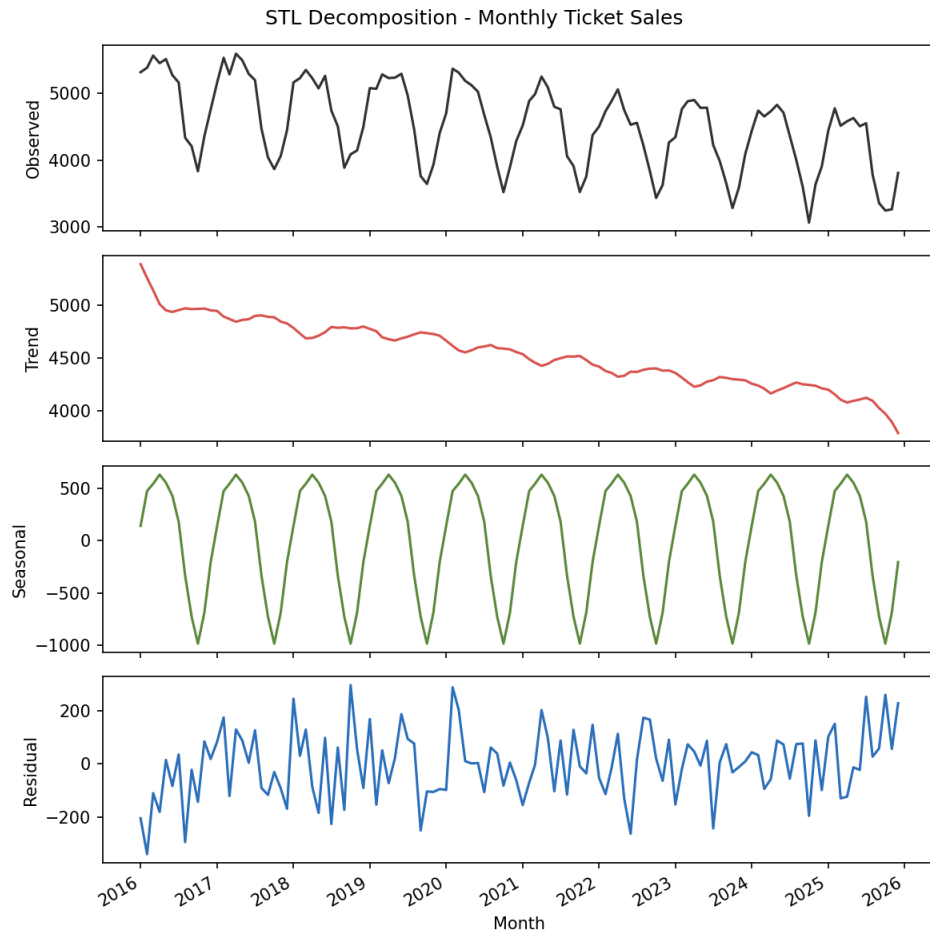
Yield peaks around 90-130 kg/ha and declines beyond that, so this range is the recommended dosage to avoid wasteful over-application.

7. An airline's revenue team suspects that ticket sales follow a strong seasonal pattern tied to holidays, but a slow underlying decline is being masked by this seasonality. Using R, apply STL decomposition to monthly ticket sales and separate the trend, seasonal, and residual components.

```
ticket_sales <- ts(read.csv("ticket_sales.csv")$Sales,
  start = c(2016, 1), frequency = 12)

decomp <- stl(ticket_sales, s.window = "periodic")
plot(decomp, main = "STL Decomposition - Monthly Ticket Sales")
```

Output:



The trend panel confirms a slow, steady decline in ticket sales once the strong yearly seasonal cycle is removed.

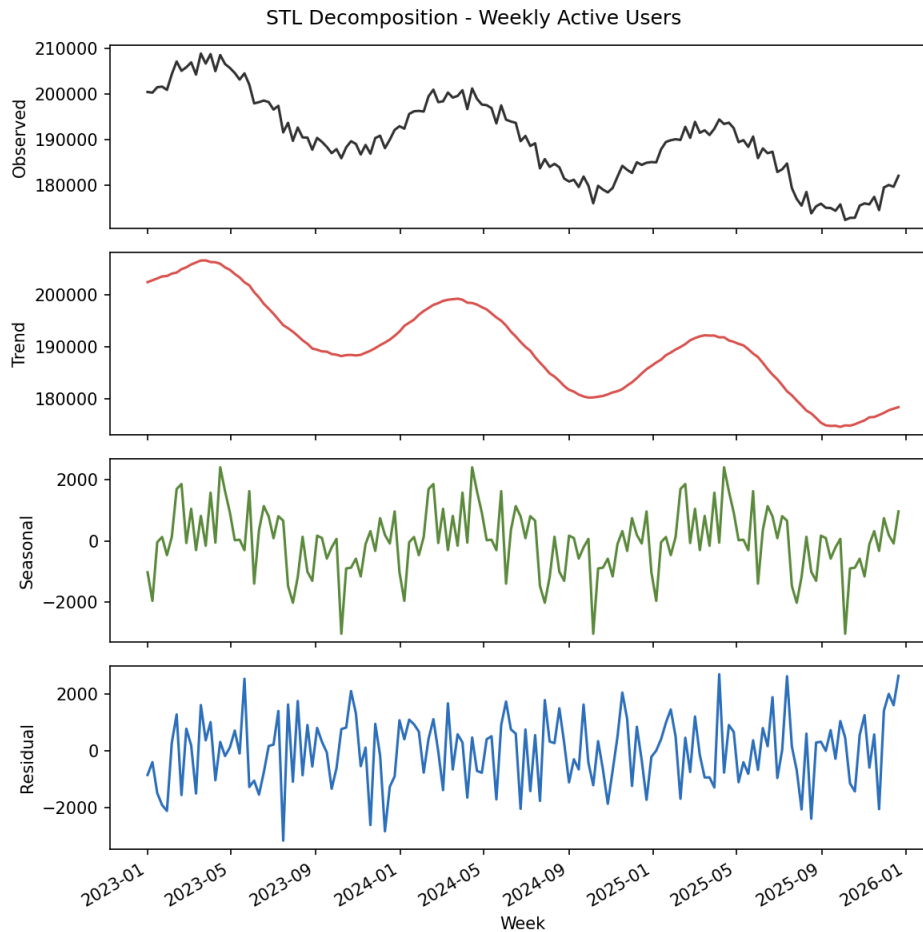
8. A streaming platform wants to know whether recent dips in weekly active users reflect a real decline or just normal seasonal viewing habits. Using Python, perform STL decomposition on the weekly user-activity data and interpret the trend component to advise the product team.

```
import pandas as pd
from statsmodels.tsa.seasonal import STL

df = pd.read_csv("weekly_active_users.csv", parse_dates=["Week"], index_col="Week")

stl = STL(df["Users"], period=52)
result = stl.fit()
result.plot()
```

Output:



The trend component drifts downward over the full three years, so the recent dips reflect a genuine decline in active users rather than normal seasonality alone.

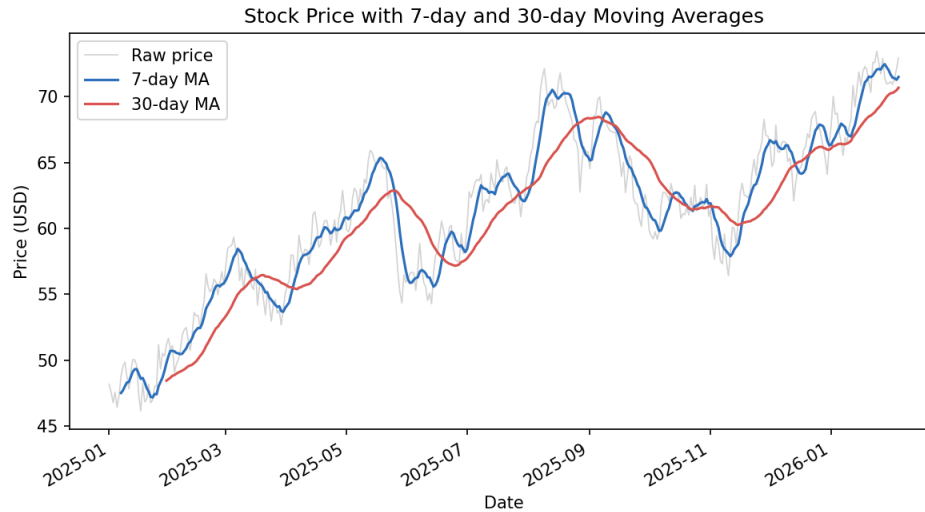
9. A stock trading firm wants to reduce short-term noise in a volatile stock's price data to spot the underlying momentum before making buy/sell calls. Using R, compute and plot a 7-day and 30-day moving average over the raw price series, and explain what the crossover between them suggests.

```
prices <- read.csv("stock_price.csv")

ma7 <- stats::filter(prices$Price, rep(1/7, 7), sides = 1)
ma30 <- stats::filter(prices$Price, rep(1/30, 30), sides = 1)

plot(prices$Date, prices$Price, type = "l", col = "gray",
      xlab = "Date", ylab = "Price (USD)")
lines(prices$Date, ma7, col = "blue")
lines(prices$Date, ma30, col = "red")
```

Output:



When the 7-day average crosses above the 30-day average, it signals building short-term upward momentum; a cross below signals weakening momentum.

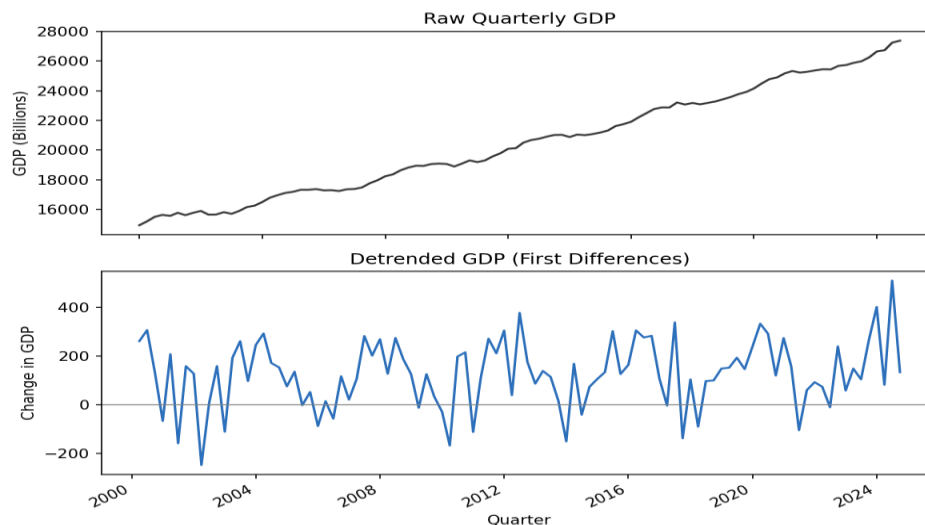
10. An economist is analyzing a country's quarterly GDP data but needs to remove the long-term growth trend to study business-cycle fluctuations in isolation. Using Python, detrend the GDP series (e.g., by differencing or regression-based detrending), plot the detrended series, and interpret the remaining fluctuations.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("gdp_quarterly.csv", parse_dates=["Quarter"])
df["GDP_diff"] = df["GDP"].diff()

fig, axes = plt.subplots(2, 1)
axes[0].plot(df["Quarter"], df["GDP"])
axes[1].plot(df["Quarter"], df["GDP_diff"])
plt.show()
```

Output:



The differenced series removes the steady growth trend, leaving cyclical ups and downs that correspond to the business cycle.